EXAM CCA-F · COMPLETE STUDY GUIDE · 2026 EDITION

Claude Certified Architect

Foundations

From zero to exam-ready: concepts, screens, real examples and 55 scenario-based practice questions with full explanations.

D1 Agentic Architecture | D2 Tools & MCP | D3 Claude Code | D4 Prompts & Structured Output | D5 Context & Reliability

Independent study guide. Not affiliated with, sponsored or endorsed by Anthropic PBC.
"Claude" is a trademark of Anthropic PBC. Exam details current as of October 2026 — always confirm on the official exam page.

How to Use This Guide

Welcome, future architect. This book takes you from "I have never called an API" to "I can justify every design decision in a production Claude system." That second sentence is exactly what the Claude Certified Architect – Foundations (CCA-F) exam measures. It does not reward memorised trivia; it rewards judgement. Every chapter is therefore built around one question: what would a senior architect do here, and why?

The structure of every domain chapter

Element	What it gives you
Learning objectives	The exact skills the exam's task statements expect from you.
Plain-English explanation	Each concept explained from first principles, no jargon assumed.
Screens (figures)	Terminal sessions, code editors and config files exactly as you will see them when you practise.
Real-world examples	Production-style cases: refunds, CI pipelines, research agents, invoice extraction.
Architect's Notes	How experienced practitioners reason about trade-offs.
Exam Traps	The distractors the exam uses to catch candidates who only skimmed the docs.
Chapter check	Scenario questions in the exam's style, with full explanations at the end of the chapter.

A word about the "screens" in this book

The figures are faithful reconstructions of what you will see in a terminal, in Claude Code and in your editor when you run the examples yourself. They are typeset (rather than pixel screenshots) so that every line stays sharp when printed and so you can copy the code exactly. Command names, file names and API fields match the official documentation at the time of writing. The best way to learn is to **type every example yourself** — the lab exercises at the end of each chapter tell you how.

EXAM TRAP — Read this first

No book, course or practice test can honestly *guarantee* a pass, and you should be suspicious of any that claims to. What this guide does guarantee is complete coverage of the five published domains and all six exam scenarios, plus a study plan that — followed honestly, with hands-on practice — gives you the best possible chance. Candidates who fail typically skipped the hands-on labs, not the reading.

ARCHITECT'S NOTE

Exam facts (fees, number of questions, registration channel) can change. Before you book, always confirm the current details on the official Anthropic certification page and in the official exam guide. Where public sources disagree, this book tells you so instead of guessing.

Contents

Chapter 0 — The Exam at a Glance . . . 5

Exam format . . . 5

The five domains and their weights . . . 5

The six scenarios . . . 6

How to read a scenario question . . . 6

Chapter 1 — Foundations for Beginners . . . 8

1.1 What Claude is — and how you talk to it . . . 8

1.2 Your first API call . . . 8

1.3 stop_reason — the most important field you will read . . . 9

1.4 Tools: how Claude acts on the world . . . 9

1.5 From chatbot to agent . . . 10

1.6 The architect's core mental model . . . 10

Chapter 2 · Domain 1 — Agentic Architecture & Orchestration . . . 12

2.1 The agentic loop, done properly . . . 12

2.2 The same loop with the Claude Agent SDK . . . 13

2.3 Workflow patterns you must recognise . . . 14

2.4 Multi-agent systems: coordinator and subagents . . . 14

2.5 Hooks: deterministic policy inside a probabilistic system . . . 15

2.6 Sessions, long-running work and task decomposition . . . 16

Lab exercise . . . 17

Chapter 2 check . . . 17

Chapter 3 · Domain 2 — Tool Design & MCP Integration . . . 19

3.1 The description is the interface . . . 19

3.2 Tool boundaries and least privilege . . . 20

3.3 Errors Claude can act on . . . 20

3.4 tool_choice . . . 21

3.5 The Model Context Protocol (MCP) . . . 21

Chapter 3 check . . . 23

Chapter 4 · Domain 3 — Claude Code Configuration & Workflows . . . 25

4.1 Your first Claude Code session . . . 25

4.2 CLAUDE.md — project memory . . . 25

4.3 Settings and permissions . . . 26

4.4 Custom commands, skills and subagents . . . 27
4.5 Plan mode and effective workflows . . . 28
4.6 Claude Code in CI/CD . . . 29
Chapter 4 check . . . 30

Chapter 5 · Domain 4 — Prompt Engineering & Structured Output . . . 32
5.1 Explicit criteria beat adjectives . . . 32
5.2 Few-shot examples . . . 33
5.3 Structured output: from "please return JSON" to guaranteed shape . . . 34
5.4 Validation–retry loops . . . 35
5.5 Cost and latency: batches and caching . . . 35
Chapter 5 check . . . 36

Chapter 6 · Domain 5 — Context Management & Reliability . . . 37
6.1 Why context is a budget, not a bucket . . . 37
6.2 Techniques for keeping what matters . . . 38
6.3 Escalation and human-in-the-loop . . . 38
6.4 Provenance: keeping sources attached to claims . . . 39
6.5 Reliability under failure . . . 39
Chapter 6 check . . . 40

Chapter 7 — The Six Exam Scenarios, Solved . . . 42
Scenario 1: Customer Support Resolution Agent . . . 42
Scenario 2: Code Generation with Claude Code . . . 43
Scenario 3: Multi-Agent Research System . . . 43
Scenario 4: Developer Productivity with Claude . . . 44
Scenario 5: Claude Code for Continuous Integration . . . 45
Scenario 6: Structured Data Extraction . . . 45

Chapter 8 — Full Mock Exam — 30 Questions . . . 47
Mock exam answer key . . . 52

Chapter 9 — Your 4-Week Pass Plan & Exam Day . . . 54
Official resources to add to your plan . . . 54
Exam-day strategy . . . 54

Appendix — Glossary . . . 56

CHAPTER 0

The Exam at a Glance

What CCA-F measures, how it is scored, and how to read a scenario question.

The Claude Certified Architect – Foundations credential launched in March 2026 as Anthropic's first technical certification. It targets people who design and ship systems built on Claude: solution architects, engineers, technical consultants and partners. "Foundations" does not mean "easy": it means the core architectural judgement every later specialisation builds on.

Exam format

Item	Detail (confirm on the official page before booking)
Questions	60 scenario-based multiple-choice questions (one best answer + three distractors)
Time	120 minutes — about 2 minutes per question
Scenarios	4 scenarios drawn at random from a published bank of 6; questions are grouped under each scenario
Scoring	Scaled score 100–1,000. **Passing score: 720.** No penalty for guessing — never leave a blank.
Delivery	Proctored, closed-book (online or test centre; registration via Anthropic's academy / partner channel)
Fee	Public sources report US $99–$125 per attempt; partner-network participants have reported waivers
Validity	Reported as 12 months, with a renewal assessment

The five domains and their weights

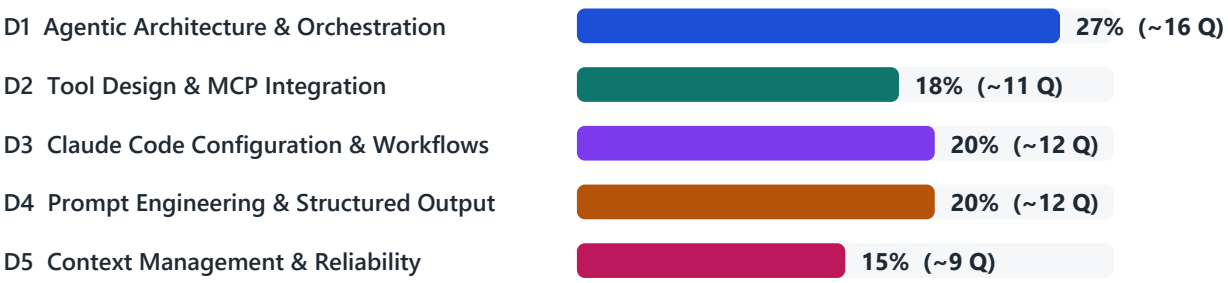


Figure 0. *Domain weights and the approximate number of questions out of 60.*

Notice that Domain 1 alone is more than a quarter of the exam, and that Domains 3 and 4 together are another 40%. If your time is limited, invest it in that order: D1 → D3/D4 → D2 → D5. But do not skip any domain: 720/1000 leaves little room for a blind spot.

The six scenarios

Every question lives inside a realistic production scenario. You will see four of these six on exam day, so you must be comfortable with all of them. Chapter 7 walks through each in depth.

#	Scenario	What you are building	Main domains
1	Customer Support Resolution Agent	Agent SDK agent with MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) aiming for high first-contact resolution	D1, D2, D5
2	Code Generation with Claude Code	A team configures Claude Code with CLAUDE.md, custom commands/skills and plan mode	D3, D5
3	Multi-Agent Research System	A coordinator delegating to search, analysis, synthesis and report subagents; cited output	D1, D2, D5
4	Developer Productivity with Claude	Built-in tools plus MCP servers to explore and change a large codebase	D2, D1, D3
5	Claude Code for Continuous Integration	Automated PR review, test generation and feedback in CI pipelines	D3, D4
6	Structured Data Extraction	JSON-schema-validated extraction from messy documents, with edge cases	D4, D5

How to read a scenario question

Exam questions are written so that **all four options sound plausible**. Three of them are usually *partially* right but violate one architectural principle. Use this four-step method on every question:

1. **Find the constraint.** Underline words like *must never*, *always*, *cost*, *latency*, *audit*, *at scale*. The constraint decides the answer.
2. **Classify the control needed.** Does the requirement need a *guarantee* (deterministic code: hooks, prerequisites, schemas, permissions) or *guidance* (prompts, examples, descriptions)? Financial, security and compliance rules need guarantees.
3. **Prefer the smallest correct fix at the right layer.** The exam loves the answer that fixes the root cause (a vague tool description, a missing schema field) over the one that adds infrastructure (a new classifier, a bigger model).
4. **Eliminate the classic distractors** (next box), then choose.

EXAM TRAP — The usual suspects

These options are wrong far more often than they are right:

- "Use a larger / more capable model" — rarely fixes a design flaw.
- "Ask the model to rate its own confidence and act on the score" — self-reported confidence is poorly calibrated.
- "Add stronger wording to the prompt (ALWAYS / NEVER / CRITICAL)" when the rule is a hard business requirement.
- "Train / fine-tune a classifier" when a better tool description or a few examples would do.
- "Give every agent every tool" — tool sprawl hurts selection accuracy and security.
- "Parse the assistant's text to decide whether the loop is done" — use `stop_reason`.

EXAM TIP

Two minutes per question is generous if you do not get stuck. Make one pass answering everything you are sure of, flag the rest, then return. Because there is no guessing penalty, every flagged question must still get an answer.

CHAPTER 1

Foundations for Beginners

The minimum you must understand before the five domains make sense: models, messages, tokens, tools and agents.

If you are new to building with large language models, this chapter is your on-ramp. Experienced developers can skim it, but do read the callouts: several exam questions hinge on these basics.

1.1 What Claude is — and how you talk to it

Claude is a family of large language models made by Anthropic. Models come in tiers — **Opus** (most capable), **Sonnet** (balanced speed/intelligence, the usual default for agents) and **Haiku** (fastest and cheapest, great for high-volume simple tasks and sub-tasks). You reach Claude in three main ways:

- **The Claude API (Messages API)** — you send a list of messages over HTTPS and receive a response. Everything else is built on this.
- **The Claude Agent SDK** — a Python/TypeScript library that runs the *agent loop* for you, with built-in tools, hooks, subagents and MCP support.
- **Claude Code** — Anthropic's agentic coding tool in the terminal, IDE and desktop; it is itself built on the same agent loop.

1.2 Your first API call

A request contains a `model`, a `max_tokens` limit, an optional `system` prompt and a list of `messages` alternating between `user` and `assistant` roles. The API is **stateless**: it remembers nothing between calls, so *you* must resend the conversation history every time.

hello_claude.py EDITOR

```python
import anthropic

client = anthropic.Anthropic()          # reads ANTHROPIC_API_KEY from the environment

response = client.messages.create(
    model="claude-sonnet-5-5",
    max_tokens=512,
    system="You are a concise assistant for an airline's support team.",
    messages=[
        {"role": "user", "content": "Can I change my flight for free within 24 hours?"}
    ],
)

print(response.stop_reason)      # -> "end_turn"
print(response.content[0].text)  # the answer text
print(response.usage)            # input_tokens / output_tokens you are billed for
```

Figure 1. *The smallest useful Messages API call. Note the three things you get back: content, stop_reason and usage.*

Terminal — python hello_claude.py TERMINAL

```
$ export ANTHROPIC_API_KEY="sk-ant-..."
$ python hello_claude.py
end_turn
Yes. Most fares can be changed without a fee within 24 hours of booking,
provided the departure is at least 7 days away. Would you like me to check
your booking?
Usage(input_tokens=38, output_tokens=41)
```

Figure 2. *Running the script.* `end_turn` *means Claude finished its answer naturally.*

BEGINNER'S CORNER

Tokens are the units models read and write — roughly ¾ of an English word each. You pay per input and output token, and every model has a **context window**: the maximum number of tokens (system prompt + history + tool results + answer) it can consider at once. Domain 5 is largely about managing this finite budget.

1.3 stop_reason — the most important field you will read

Every response tells you *why* Claude stopped generating. Your code must branch on this field, never on the wording of the text.

stop_reason	Meaning	What your code should do
`end_turn`	Claude finished its turn naturally	Return the answer / end the loop
`tool_use`	Claude wants you to run one or more tools	Execute the tools, send back `tool_result` blocks, call again
`max_tokens`	Output hit your `max_tokens` limit	Answer is truncated: raise the limit, continue, or chunk the task
`stop_sequence`	A custom stop string was generated	Handle according to your protocol
`pause_turn`	A long server-side tool turn was paused	Send the response back to let Claude continue
`refusal`	Claude declined for safety reasons	Do not retry blindly; route or report

1.4 Tools: how Claude acts on the world

On its own, a model can only produce text. **Tool use** (function calling) lets you describe functions — with a `name`, a natural-language `description` and a JSON Schema `input_schema` — that Claude may ask to call. Claude never executes anything itself: it returns a `tool_use` block, *your code* runs the function, and you return the result in a `tool_result` block.

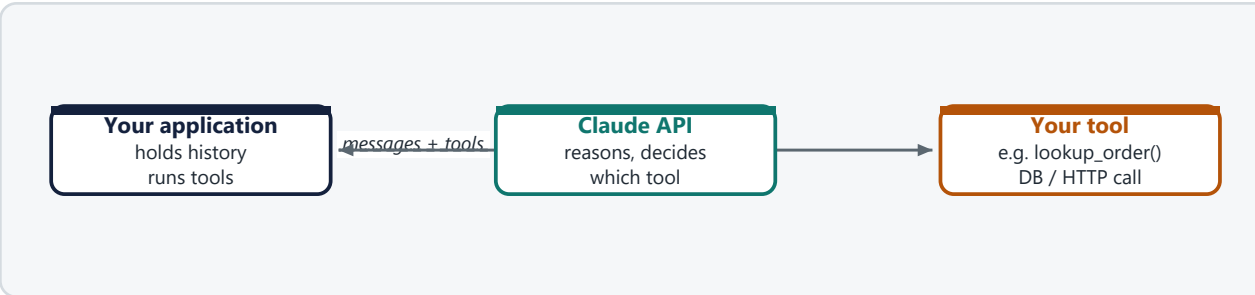


Figure 3. *Claude decides; your application executes. The dashed arrow is the* `tool_use` *response coming back.*

1.5 From chatbot to agent

An **agent** is simply a model calling tools *in a loop* until a task is done: think → act (tool) → observe (result) → think again. A **workflow** is a fixed, code-defined sequence of LLM calls. Architects choose between them deliberately:

	Workflow (predefined path)	Agent (model-directed loop)
Control flow	Your code decides the steps	Claude decides the next step
Best when	Steps are known and stable; predictability, cost and audit matter	Steps are open-ended or depend on what is discovered
Examples	Prompt chaining, routing, parallel fan-out	Debugging a codebase, open-ended research, support cases
Risk	Rigid when inputs vary	Higher cost/latency, needs guardrails

ARCHITECT'S NOTE

Anthropic's own guidance is to **start with the simplest thing that works** — often a single well-prompted call — and add agentic autonomy only when the task truly needs it. On the exam, an answer that adds an agent or extra agents "for flexibility" when a deterministic workflow satisfies the requirement is usually a distractor.

1.6 The architect's core mental model

If you remember one idea from this book, make it this one. Every requirement in a Claude system can be enforced at one of two levels:

Probabilistic controls (guidance)	Deterministic controls (guarantees)
System prompts, instructions, few-shot examples, tool descriptions, CLAUDE.md guidance	Code that runs regardless of what the model says: hooks (PreToolUse), prerequisite checks, permission rules, JSON schema validation, `tool_choice`, allow/deny tool lists, sandboxing
Followed *very often* — great for style, judgement, preferences	Followed *always* — required for money, security, compliance, ordering

KEY TAKEAWAYS

- The API is stateless: you own the conversation history.
- Branch on `stop_reason`, not on text.
- Claude requests tools; your code executes them.
- "Must never / must always" requirements → deterministic controls. Preferences → prompts.

CHAPTER 2 · DOMAIN 1

Exam weight 27%

Agentic Architecture & Orchestration

Agent loops, multi-agent systems, hooks, sessions and task decomposition.

Learning objectives

- Implement a correct agentic loop driven by `stop_reason`, including tool-result handling and safety limits.
- Choose between a single agent, a workflow, and a coordinator–subagent design, and justify the choice.
- Pass context explicitly to subagents and keep each subagent's tools and scope narrow.
- Use Agent SDK hooks to enforce deterministic policy (PreToolUse, PostToolUse, Stop, etc.).
- Manage long-running work: sessions, resumption, forking and task decomposition.

2.1 The agentic loop, done properly

Here is the loop at the heart of every agent. Read it slowly — the exam tests each highlighted line.

support_agent/loop.py EDITOR

```python
def run_agent(user_msg: str, max_turns: int = 15):
    messages = [{"role": "user", "content": user_msg}]
    for turn in range(max_turns):                    # safety limit, not the exit condition
        resp = client.messages.create(model=MODEL, max_tokens=2048,
                                      system=SYSTEM, tools=TOOLS, messages=messages)
        messages.append({"role": "assistant", "content": resp.content})   # keep tool_use blocks!

        if resp.stop_reason == "end_turn":           # Claude is done -> exit
            return final_text(resp)
        if resp.stop_reason == "tool_use":
            results = []
            for block in resp.content:
                if block.type == "tool_use":
                    out = dispatch(block.name, block.input)      # YOUR code runs the tool
                    results.append({"type": "tool_result",
                                    "tool_use_id": block.id,     # must match the request
                                    "content": out.text,
                                    "is_error": out.is_error})
            messages.append({"role": "user", "content": results})  # all results, one message
            continue
        if resp.stop_reason == "max_tokens":
            raise TruncatedOutput(turn)              # handle explicitly, never ignore
    escalate("turn budget exhausted", messages)      # fail safe, not fail silent
```

Figure 4. *A production-grade agent loop. Highlighted lines are the ones exam questions are built around.*

Line-by-line: what an architect checks

- **The exit condition is** `stop_reason`, not "the text contains DONE" and not "no more tool calls in the text". Parsing natural language to decide control flow is a classic wrong answer.
- **The assistant turn is appended in full**, including its `tool_use` blocks. Dropping them breaks the pairing between `tool_use.id` and `tool_result.tool_use_id`, and the API will reject the next request.

- **Claude can request several tools in one turn** (parallel tool use). Return *all* their results in a *single* user message.
- `max_turns` **is a safety net**, not the normal termination mechanism. When it trips, escalate or report — never silently return a half-finished answer as if it were complete.
- **Errors are returned to Claude as results** (`is_error: true`) so it can adapt — retry with different input, try another tool, or explain the problem to the user.

> **EXAM TRAP**
> Options such as "stop the loop when the assistant's message contains no question", "check whether the response text says the task is complete", or "always run exactly N iterations" are wrong. The model signals completion through `stop_reason == "end_turn"`.

2.2 The same loop with the Claude Agent SDK

The Agent SDK runs this loop for you, with built-in tools (Read, Write, Edit, Bash, Grep, Glob, WebSearch, WebFetch…), permissions, hooks, subagents, sessions and MCP. You configure *policy*; the SDK handles mechanics.

support_agent/main.py EDITOR

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

options = ClaudeAgentOptions(
    system_prompt=SUPPORT_POLICY,
    mcp_servers={"crm": crm_server},                    # custom tools via MCP
    allowed_tools=["mcp__crm__get_customer", "mcp__crm__lookup_order",
                   "mcp__crm__process_refund", "mcp__crm__escalate_to_human"],
    max_turns=15,
)

async def main():
    async for message in query(prompt="Order #A-7731 arrived broken, I want a refund",
                               options=options):
        print(message)          # assistant text, tool calls, final ResultMessage

asyncio.run(main())
```

Figure 5. *The Agent SDK version: you declare tools, limits and policy; the SDK runs the loop.*

> **BEGINNER'S CORNER**
> MCP tool names in the SDK and Claude Code follow the pattern `mcp__<server>__<tool>`. Listing them in `allowed_tools` is how you scope exactly which tools an agent may use.

2.3 Workflow patterns you must recognise

Pattern	How it works	Use it when…
Prompt chaining	Output of step 1 feeds step 2, with code "gates" between steps	A task splits cleanly into fixed sequential steps (outline → draft → check)
Routing	A classifier step sends the input to a specialised prompt/model	Distinct input categories need different handling (billing vs technical)
Parallelisation	Independent sub-tasks run at once (sectioning) or the same task runs several times (voting)	Speed matters or you want multiple perspectives/guardrails
Orchestrator–workers	A lead model decides sub-tasks dynamically and delegates	Sub-tasks cannot be known in advance (research, multi-file changes)
Evaluator–optimiser	One call generates, another critiques against criteria, loop	Clear evaluation criteria exist and iteration measurably helps

2.4 Multi-agent systems: coordinator and subagents

In a coordinator–subagent (hub-and-spoke) design, a lead agent breaks the job down and delegates to specialised subagents. Each subagent runs in its **own fresh context window**, does its work, and returns a **condensed result** to the coordinator.

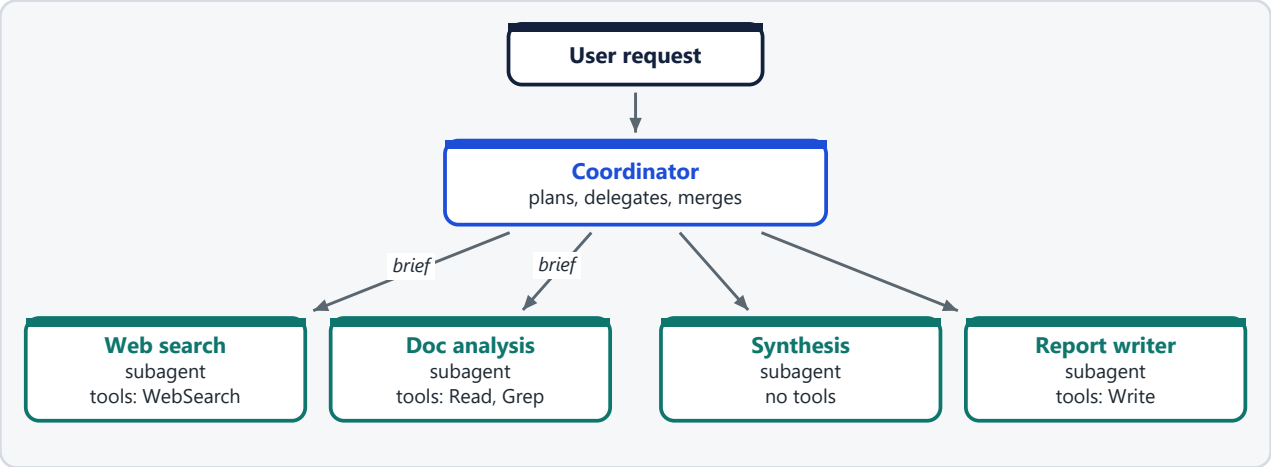


Figure 6. *Hub-and-spoke: each subagent gets a focused brief and only the tools its role needs.*

Why use subagents?

- **Context isolation** — a subagent can read 40 files and return a 300-word summary, keeping the coordinator's context clean.
- **Parallelism** — independent searches run simultaneously.
- **Specialisation** — each has a focused prompt and a minimal tool set, which improves tool selection and security.

The rule examiners care about most: context is not inherited

A subagent does **not** see the coordinator's conversation. Whatever it needs — the goal, constraints, relevant facts, the expected output format, what *not* to do — must be written into the task brief. Vague delegation ("research the topic") leads to duplicated work, wrong scope and missing citations.

research/agents.py EDITOR

```python
from claude_agent_sdk import ClaudeAgentOptions, AgentDefinition

options = ClaudeAgentOptions(
    allowed_tools=["Task", "Read", "Write"],          # coordinator can delegate via the Task tool
    agents={
        "web-researcher": AgentDefinition(
            description="Finds and summarises primary sources on ONE sub-question. "
                        "Use for any fact that needs a web citation.",
            prompt="Return at most 8 findings. Each finding: claim, quote, URL, date. "
                   "Flag conflicting sources instead of choosing one.",
            tools=["WebSearch", "WebFetch"],          # least privilege
            model="haiku"),                           # cheaper model for a narrow task
        "synthesiser": AgentDefinition(
            description="Merges findings into a structured outline. Never searches.",
            prompt="Preserve every citation. Mark claims supported by only one source.",
            tools=[]),
    },
)
```

Figure 7. *Programmatic subagent definitions. The* `description` *tells the coordinator when to delegate;* `tools` *enforces least privilege.*

REAL-WORLD EXAMPLE — From monolith to hub-and-spoke

A consulting firm built a market-research agent. Version 1 gave one agent all tools and a 6-page prompt; reports were slow and citations were often lost. Version 2 used a coordinator plus three subagents with explicit briefs ("sub-question, time window, max 8 findings, return claim+quote+URL"). Each subagent returned structured findings; the synthesiser was forbidden from searching. Result: parallel searches, shorter coordinator context, and citations that survived to the final report because the *format required them*.

EXAM TRAP

- "Let subagents share the coordinator's full conversation history so they have all the context" — wasteful and defeats isolation; pass a curated brief.
- "Subagents should talk to each other directly" — in the hub-and-spoke model, communication flows through the coordinator, which keeps a single source of truth.
- "Give all subagents all tools for flexibility" — violates least privilege and degrades tool choice.

ARCHITECT'S NOTE

Multi-agent systems cost more tokens (often several times a single agent). Use them when the task is parallelisable or exceeds one context window — not for simple, sequential tasks where one agent with good tools suffices.

2.5 Hooks: deterministic policy inside a probabilistic system

Hooks are your code, executed by the SDK (or Claude Code) at fixed lifecycle points. Because they run every time, they are how you turn a business rule into a **guarantee**.

Hook event	Fires…	Typical use
`PreToolUse`	Before a tool executes; can allow, deny or modify the call	Block refunds above a limit, block identity-less refunds, deny dangerous shell commands

Hook event	Fires…	Typical use
`PostToolUse`	After a tool returns	Normalise/trim results, redact PII, audit logging, run a linter after edits
`UserPromptSubmit`	When a user prompt is submitted	Inject context, block disallowed requests
`Stop` / `SubagentStop`	When the agent / a subagent is about to finish	Verify completion criteria (e.g. tests pass) before allowing the stop
`PreCompact`	Before context compaction	Save critical facts to durable storage
`SessionStart` / `SessionEnd`	Session boundaries	Load case facts, write summaries

support_agent/hooks.py EDITOR

```python
from claude_agent_sdk import ClaudeAgentOptions, HookMatcher

async def refund_guard(input_data, tool_use_id, context):
    args = input_data["tool_input"]
    case = load_case(input_data["session_id"])
    if not case.identity_verified:
        return deny("Identity not verified. Call get_customer and verify first.")
    if args["amount"] > 500:
        return deny("Refunds over $500 require a human. Use escalate_to_human.")
    return {}                                          # allow

def deny(reason):
    return {"hookSpecificOutput": {"hookEventName": "PreToolUse",
            "permissionDecision": "deny", "permissionDecisionReason": reason}}

options = ClaudeAgentOptions(hooks={
    "PreToolUse": [HookMatcher(matcher="mcp__crm__process_refund", hooks=[refund_guard])]})
```

Figure 8. *A PreToolUse hook enforcing two business rules. The denial reason goes back to Claude so it can recover gracefully.*

EXAM TIP

When a question says a rule *must* hold ("refunds must never be issued before identity verification", "the agent must never write outside /src"), the answer is almost always a **hook, permission rule or programmatic prerequisite** — not a better prompt. When the question is about *quality* or *style*, prompting is the right tool.

2.6 Sessions, long-running work and task decomposition

- **Sessions** persist a conversation so work can **resume** later (`resume=<session_id>` in the SDK, `claude --resume` / `--continue` in Claude Code). **Forking** a session lets you explore an alternative approach without polluting the original.
- **Resume vs. fresh start:** resume when prior context is still valid; start fresh with a *structured summary* when the old context is stale (files changed) or bloated. Re-feeding stale tool results is a source of errors.
- **Decomposition:** for large tasks, split by independent units (per file, per document, per sub-question). Use fixed pipelines when the steps are known, dynamic orchestrator–worker decomposition when they are not.
- **Checkpoint state outside the context window** — progress files, task lists, databases — so a crash or compaction does not lose the plan.

Lab exercise

1. Implement the loop in Figure 4 with two fake tools (`get_weather`, `get_time`). Print `stop_reason` on every turn.
2. Ask a question needing both tools and observe parallel `tool_use` blocks in one response.
3. Make one tool raise an error; return `is_error: true` and watch Claude recover.
4. Port it to the Agent SDK and add a PreToolUse hook that denies one tool. Read the denial reason in the transcript.

Chapter 2 check

Q1. Your support agent sometimes stops after calling `lookup_order` and returns nothing to the user. The loop exits when the assistant's text does not contain a question mark. What is the best fix?

A. Increase `max_tokens` to 8,192
B. Run a second model to judge whether the conversation is complete
C. Add "always end with a summary" to the system prompt
D. Drive the loop with `stop_reason`: continue on `tool_use`, exit on `end_turn`

Q2. Company policy: refunds above $500 must be approved by a human. In testing the agent occasionally refunds $800 despite a clear instruction in the system prompt. What should the architect do?

A. Switch to the most capable model
B. Repeat the rule three times in the prompt in capital letters
C. Ask Claude to output a confidence score before each refund
D. Add a PreToolUse hook on `process_refund` that denies amounts above $500 and directs Claude to escalate

Q3. A coordinator delegates "research competitor pricing" to a subagent. The subagent returns generic information and misses the client's region and date range that were discussed earlier with the coordinator. Why?

A. The subagent's model is too small
B. Subagents do not inherit the coordinator's conversation; the brief must include the constraints
C. The subagent's `max_turns` was too low
D. The coordinator should have used parallel tool calls

Q4. Which situation most justifies a coordinator with multiple subagents instead of a single agent?

A. A fixed three-step pipeline: extract, validate, store
B. Answering FAQ questions from a small knowledge base
C. Translating one short email
D. An open-ended research question requiring many independent searches whose results would overflow one context window

Q5. An agent hits `max_turns` mid-task. What is the most appropriate behaviour?

A. Return the last assistant message as the final answer
B. Surface the incomplete state: escalate or report what was done and what remains
C. Delete the history and retry from scratch
D. Silently restart the loop with a higher limit

Chapter 2 answers & explanations

Q1 — Answer D. Control flow must depend on the structured `stop_reason` field. After a tool call the stop reason is `tool_use`; the loop must run the tool, return the result and call the model again. Prompt wording (A) and a judge model (D) are indirect patches; `max_tokens` (C) is unrelated.

Q2 — Answer D. A financial rule that must always hold needs a deterministic control. The hook runs on every call regardless of the model's reasoning, and its denial message steers Claude to `escalate_to_human`.

Q3 — Answer B. Subagents start with a fresh context. Anything they need — region, dates, output format — must be passed explicitly in the task description.

Q4 — Answer D. Multi-agent designs pay off when work is parallelisable and context-heavy. Fixed pipelines (B) are workflows; A and D are single-call tasks.

Q5 — Answer B. The turn limit is a safety net. Hitting it means the task is incomplete; the system should fail visibly and safely, preserving state for a human or a resumed run.

CHAPTER 3 · DOMAIN 2

Exam weight 18%

Tool Design & MCP Integration

Descriptions that steer selection, structured errors, least privilege, and the Model Context Protocol.

Learning objectives

- Write tool names, descriptions and schemas that make Claude choose the right tool reliably.
- Design tool boundaries: when to split, merge or remove tools; scope tools per agent role.
- Return structured, actionable errors that distinguish retryable from non-retryable failures.
- Explain MCP's architecture (hosts, clients, servers; tools, resources, prompts; transports) and configure servers.
- Use `tool_choice` correctly and handle authentication and secrets safely.

3.1 The description is the interface

Claude chooses tools by reading their **names, descriptions and schemas**. For a model, the description is the documentation, the API contract and the usage guide all in one. When an agent picks the wrong tool, the root cause is almost always an ambiguous or overlapping description — and the fix is to rewrite it, not to add a routing classifier.

tools_before.json — ambiguous EDITOR

```
[
  {"name": "get_customer", "description": "Gets customer info",
   "input_schema": {"type": "object", "properties": {"query": {"type": "string"}}}},
  {"name": "lookup_order", "description": "Gets info about a customer order",
   "input_schema": {"type": "object", "properties": {"query": {"type": "string"}}}}
]
```

Figure 9. *Before: two near-duplicate descriptions and a free-text* `query` *parameter. Claude will confuse them.*

tools_after.json — disambiguated EDITOR

```
[
  {"name": "get_customer",
   "description": "Look up a customer's PROFILE and verify identity. Use FIRST in every case, before any order or refund action. Inpu
   "input_schema": {"type": "object",
     "properties": {"email_or_phone": {"type": "string", "description": "Exactly as given by the customer"}},
     "required": ["email_or_phone"]}},
  {"name": "lookup_order",
   "description": "Fetch ONE order by order ID (format A-1234) for an already-identified customer_id. Use when the customer mentions
   "input_schema": {"type": "object",
     "properties": {"customer_id": {"type": "string"},
                    "order_id": {"type": "string", "pattern": "^A-[0-9]{4}$"}},
     "required": ["customer_id", "order_id"]}}
]
```

Figure 10. *After: each description says what it does, when to use it, what it needs, what it returns — and what it does NOT do.*

A checklist for great tool definitions

- **What & when:** purpose, the situations to use it in, and the situations *not* to (boundaries with sibling tools).
- **Inputs:** precise types, formats, enums, examples, `required` fields. Prefer specific parameters over a free-text `query`.
- **Outputs:** what comes back, so Claude can plan the next step.
- **Names:** unambiguous and consistently namespaced (`crm_get_customer`, `crm_lookup_order`).
- **Return high-signal data:** human-readable fields over cryptic IDs; trim huge payloads; paginate.

> **EXAM TRAP**
> When the scenario says "the agent frequently calls `lookup_order` when it should call `get_customer`", the best first fix is **improving the tool descriptions** (and possibly the parameter names). Adding a keyword-based pre-router, fine-tuning, or few-shot examples in the system prompt are heavier answers that do not address the root cause.

3.2 Tool boundaries and least privilege

More tools is not better. Each extra tool adds tokens to every request and another chance to pick the wrong one. Architects shape the tool set around **workflows**, not around the raw API.

Design choice	Prefer	Avoid
Granularity	`schedule_meeting` that checks availability and books	Exposing `list_users`, `list_events`, `create_event` separately when they are always chained
Scope per role	Read-only tools for a reviewer agent; write tools only where needed	Every agent sees every tool
Risky operations	Separate, clearly named, guarded by hooks/permissions	A generic `execute_sql(query)` for a support agent
Number of tools	A focused set per agent (rule of thumb: keep it small, split by role if it grows)	Dozens of overlapping tools in one agent

3.3 Errors Claude can act on

A tool that returns `"Error"` or an empty string leaves Claude guessing. Return a **structured error** in the `tool_result` with `is_error: true`, saying what failed, whether retrying can help, and what to do instead.

crm_server/errors.py EDITOR

```python
def tool_error(category, message, retryable, suggestion=None):
    return {"is_error": True, "content": json.dumps({
        "error_category": category,      # validation | not_found | permission | transient | business_rule
        "message": message,
        "retryable": retryable,
        "suggestion": suggestion})}

# transient: worth retrying (ideally inside the tool with backoff first)
tool_error("transient", "CRM timed out after 10s", True, "Retry once; if it fails again, apologise and escalate.")
# validation: fix the input, do not retry as-is
tool_error("validation", "order_id 'A77' does not match ^A-[0-9]{4}$", False, "Ask the customer for the full order ID.")
# business rule: never retry, change course
tool_error("business_rule", "Order is outside the 30-day refund window", False, "Offer store credit or escalate_to_human
```

Figure 11. *Structured errors: category, retryability and a next step. Notice that* `not_found` *(an empty result) is not the same as a failure.*

ARCHITECT'S NOTE

Distinguish **"the lookup failed"** from **"the lookup succeeded and found nothing"**. Returning an empty list as an error makes Claude retry pointlessly; returning a timeout as "no results" makes Claude tell a customer their order does not exist. Both are classic exam scenarios.

3.4 tool_choice

Value	Behaviour	When to use
`{"type": "auto"}`	Claude decides whether and which tool to call (default with tools)	Normal agent operation
`{"type": "any"}`	Claude must call *some* tool	You need an action, never a plain-text reply
`{"type": "tool", "name": "X"}`	Claude must call tool X	Forcing structured output through an extraction tool; a mandatory first step
`{"type": "none"}`	Claude may not call tools	A final summarisation turn

3.5 The Model Context Protocol (MCP)

MCP is an open standard for connecting AI applications to tools and data — often described as "USB-C for AI". Instead of writing a custom integration for every app, you build an **MCP server** once and any MCP client (Claude Code, the Agent SDK, Claude Desktop, other hosts) can use it.

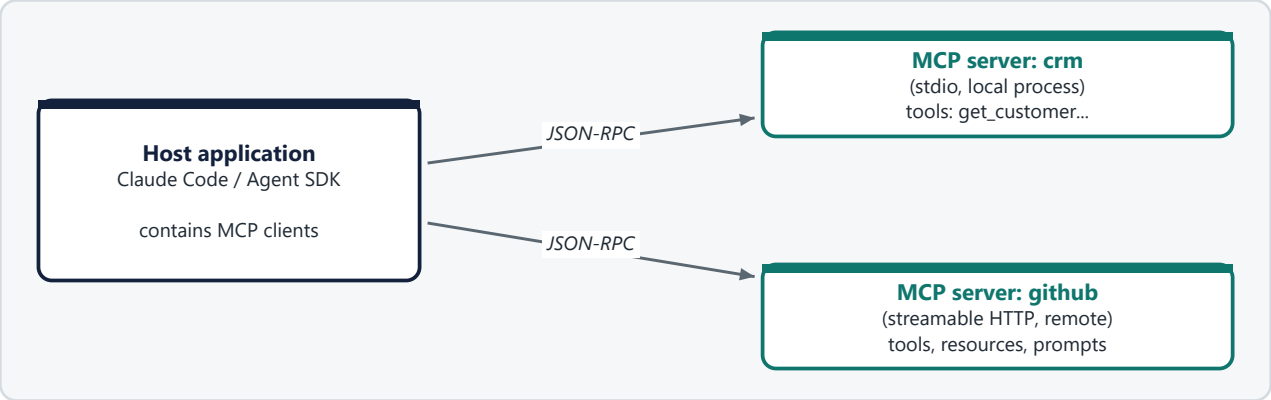


Figure 12. *One host, one client connection per server. Local servers usually use stdio; remote servers use streamable HTTP.*

Primitive	Controlled by	What it is	Example
Tools	The model	Functions Claude can call to act or fetch	`create_issue`, `run_query`
Resources	The application	Read-only data exposed by URI to be loaded as context	`file:///docs/policy.md`, a DB schema
Prompts	The user	Reusable prompt templates, often surfaced as slash commands	`/review-pr`

Configuring MCP servers in Claude Code

Terminal — adding MCP servers TERMINAL

```
$ claude mcp add crm --scope project -- python crm_server.py
Added stdio MCP server crm to project config (.mcp.json)
$ claude mcp add --transport http github https://api.githubcopilot.com/mcp/ --scope user
Added HTTP MCP server github to user config
$ claude mcp list
crm:    python crm_server.py - ✓ Connected
github: https://api.githubcopilot.com/mcp/ (HTTP) - ✓ Connected
```

Figure 13. *Scopes:* `local` *(just you, this project — the default),* `project` *(shared via* `.mcp.json` *in git),* `user` *(you, all projects).*

.mcp.json (committed to the repo) EDITOR

```
{
  "mcpServers": {
    "crm": {
      "command": "python",
      "args": ["crm_server.py"],
      "env": { "CRM_API_KEY": "${CRM_API_KEY}" }
    },
    "jira": {
      "type": "http",
      "url": "https://jira.internal.example.com/mcp",
      "headers": { "Authorization": "Bearer ${JIRA_TOKEN}" }
    }
  }
}
```

Figure 14. *Project-scoped MCP config. Secrets are referenced as environment variables — never committed.*

A minimal MCP server

crm_server.py EDITOR

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("crm")

@mcp.tool()
def lookup_order(customer_id: str, order_id: str) -> dict:
    """Fetch ONE order by ID (format A-1234) for an identified customer.
    Use when the customer mentions a specific order. Returns status, items, amount."""
    return db.orders.get(customer_id, order_id)

@mcp.resource("policy://refunds")
def refund_policy() -> str:
    """The current refund policy text."""
    return open("policies/refunds.md").read()

if __name__ == "__main__":
    mcp.run()          # stdio transport by default
```

Figure 15. *With the Python MCP SDK, the function signature becomes the input schema and the docstring becomes the tool description.*

EXAM TIP

- Team-shared server → **project scope** (`.mcp.json`) with `${ENV_VAR}` secrets.
- Personal tool across all your projects → **user scope**.
- Prefer a maintained, existing MCP server over writing your own for standard systems (GitHub, Jira…) unless you need custom behaviour.
- An MCP server's tools still need good descriptions — MCP changes the transport, not the rules of tool design.

Chapter 3 check

Q1. Logs show the agent calls `search_kb` when users ask about their invoices, instead of `get_invoice`. Both descriptions are one line long. What is the most effective first step?

A. Fine-tune a model on correct tool choices

B. Remove `search_kb`

C. Add a keyword router that sends any message containing "invoice" to `get_invoice`

D. Rewrite both descriptions to state purpose, when to use, inputs, outputs and boundaries vs. each other

Q2. An inventory tool times out. It currently returns `[]`, and the agent tells customers the product doesn't exist. Best fix?

A. Instruct Claude never to trust empty results

B. Return a structured error with `is_error: true`, category `transient`, `retryable: true`

C. Raise an exception that crashes the agent loop

D. Return a fake "in stock" result to avoid upsetting customers

Q3. A team wants every developer's Claude Code to use the company's internal Jira MCP server, without anyone committing credentials. Which setup is best?

A. Each developer adds it with local scope and pastes the token

B. Put the token in CLAUDE.md

C. Add it to the user scope of one admin's machine

D. Add it to `.mcp.json` at project scope with the token referenced as `${JIRA_TOKEN}`

Q4. You need Claude to always return data through your `record_extraction` tool rather than as prose. Which setting enforces this?

A. `tool_choice: {"type": "auto"}`

B. A system prompt saying "always use the tool"

C. `tool_choice: {"type": "none"}`

D. `tool_choice: {"type": "tool", "name": "record_extraction"}`

Q5. Which MCP primitive is best for exposing a large, read-only product catalogue for the application to load as context?

A. A tool

B. A prompt

C. A hook

D. A resource

Chapter 3 answers & explanations

Q1 — Answer D. Tool selection is driven by descriptions. Clear, differentiated descriptions fix the root cause with the smallest change; routers and fine-tuning add complexity and brittleness.

Q2 — Answer B. An empty result means "found nothing". A timeout is a failure and must be reported as such, with retry guidance, so Claude can retry or explain accurately.

Q3 — Answer D. Project scope shares the configuration through version control; environment-variable expansion keeps secrets out of the repository.

Q4 — Answer D. Forcing a specific tool guarantees a `tool_use` block matching that tool's schema; `auto` and prompts leave it to the model.

Q5 — Answer D. Resources are application-controlled, URI-addressed read-only data. Tools are model-invoked actions; prompts are user-invoked templates; hooks are not an MCP primitive.

CHAPTER 4 · DOMAIN 3

Exam weight 20%

Claude Code Configuration & Workflows

CLAUDE.md memory, rules, settings and permissions, commands and skills, plan mode, and Claude Code in CI/CD.

Learning objectives

- Place instructions at the right level of the CLAUDE.md hierarchy and use imports and path-scoped rules.
- Configure permissions and settings at user, project and local scope.
- Create custom slash commands and skills; know when to use each, and when to use subagents.
- Use plan mode and other workflows (explore → plan → code → commit) appropriately.
- Run Claude Code non-interactively (`-p`) in CI/CD with structured output and safe permissions.

4.1 Your first Claude Code session

Terminal — ~/projects/shop-api TERMINAL

```
$ npm install -g @anthropic-ai/claude-code
$ cd ~/projects/shop-api && claude
> /init
● I'll analyse the codebase and create a CLAUDE.md file.
  └ Read package.json, tsconfig.json, src/ (42 files), .github/workflows/ci.yml
● Created CLAUDE.md with build/test commands, architecture overview and conventions.
> add input validation to POST /orders and write tests for it
● I'll look at the existing route and validation patterns first.
  └ Read src/routes/orders.ts  └ Grep "zod" (6 matches)
● Update(src/routes/orders.ts)  ... Do you want to make this edit? 1. Yes  2. Yes, don't ask again  3. No
```

Figure 16. *A typical session: `/init` bootstraps project memory; Claude explores before editing and asks permission for changes.*

4.2 CLAUDE.md — project memory

CLAUDE.md files are loaded into Claude Code's context automatically. They hold the knowledge a new teammate would need: build and test commands, architecture, conventions, gotchas. The key exam skill is **putting each instruction at the right level**.

Level	Location	Shared with	Put here…
Enterprise / managed policy	OS-level managed path set by IT	Everyone in the organisation	Security and compliance rules
User	`~/.claude/CLAUDE.md`	Only you, all projects	Personal preferences (tone, your editor habits)
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	The team (committed to git)	Build/test commands, architecture, team conventions
Subdirectory	`./packages/api/CLAUDE.md`	The team	Rules for that part of the tree; loaded when Claude works there

Level	Location	Shared with	Put here...
Local (personal, per project)	`CLAUDE.local.md` (git-ignored) or imports from your home dir	Only you	Your sandbox URLs, personal test data

CLAUDE.md EDITOR

```
# Shop API

## Commands
- Build: `npm run build`   - Test one file: `npm test -- path/to/file.test.ts`
- Lint before committing: `npm run lint:fix`

## Architecture
- Express + TypeScript. Routes in src/routes, business logic in src/services.
- All DB access goes through src/repositories (never query Prisma from a route).

## Conventions
- Validate every request body with zod schemas in src/schemas.
- IMPORTANT: money is stored in integer cents. Never use floats for amounts.

@docs/api-conventions.md
@~/.claude/my-shop-api-notes.md
```

Figure 17. *A good project CLAUDE.md: short, specific, actionable.* `@path` *lines import other files (including from your home directory).*

Path-specific rules

For instructions that only apply to certain files, use rule files in `.claude/rules/` with a `paths` pattern in their frontmatter. They load only when Claude works on matching files, keeping context lean.

.claude/rules/testing.md EDITOR

```
---
paths:
  - "**/*.test.ts"
  - "tests/**"
---
# Testing rules
- Use the factories in tests/factories; never hand-build Order objects.
- Each test must be independent: no shared mutable state between tests.
```

Figure 18. *A path-scoped rule. Prefer this over stuffing every rule into one giant CLAUDE.md.*

EXAM TRAP

- A *team* convention placed in `~/.claude/CLAUDE.md` will not reach teammates — it belongs in the project file.
- A *personal* preference committed to the project CLAUDE.md imposes it on the whole team.
- A huge CLAUDE.md wastes context on every request and dilutes important rules. Keep it concise; split with imports and path rules.
- CLAUDE.md is guidance. If a rule must be *enforced* (never edit `.env`, never run `rm -rf`), use permissions or hooks.

4.3 Settings and permissions

.claude/settings.json (team, committed) EDITOR

```
{
  "permissions": {
    "allow": ["Bash(npm run test:*)", "Bash(npm run lint:*)", "Read(src/**)", "Edit(src/**)"],
    "ask":   ["Bash(git push:*)"],
    "deny":  ["Read(./.env)", "Read(./secrets/**)", "Bash(curl:*)", "Bash(rm -rf:*)"]
  },
  "hooks": {
    "PostToolUse": [{ "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write" }] }]
  }
}
```

Figure 19. *Allow / ask / deny rules plus a hook that formats every edited file. Deny rules take precedence over allow rules.*

File	Scope	Committed?
`~/.claude/settings.json`	You, every project	No
`.claude/settings.json`	Everyone on the project	Yes
`.claude/settings.local.json`	You, this project	No (git-ignored)
Managed policy settings	Whole organisation; cannot be overridden	Deployed by IT

Permission modes you can cycle with `Shift+Tab`: *default* (asks before edits and commands), *accept edits* (auto-approves file edits), and *plan mode* (read-only analysis and planning, no changes).

4.4 Custom commands, skills and subagents

.claude/commands/fix-issue.md EDITOR

```
---
description: Fix a GitHub issue end-to-end
argument-hint: [issue-number]
allowed-tools: Bash(gh issue view:*), Read, Edit, Bash(npm test:*)
---
Fix GitHub issue #$ARGUMENTS.
1. Run `gh issue view $ARGUMENTS` and restate the problem.
2. Find the relevant code, then write a failing test that reproduces it.
3. Fix the code until the test passes; run the full test suite.
4. Summarise the root cause and the change.
```

Figure 20. *A project slash command, invoked as `/fix-issue 482`. Committing it in `.claude/commands/` shares it with the team.*

.claude/skills/release-notes/SKILL.md EDITOR

```
---
name: release-notes
description: Writes customer-facing release notes from merged PRs. Use when the user asks
  for release notes, a changelog or "what shipped".
allowed-tools: Bash(git log:*), Bash(gh pr list:*), Read
---
# Release notes
1. Collect merged PRs since the last tag.
2. Group by Features / Fixes / Breaking changes; plain language, no PR jargon.
3. Follow the template in ./template.md (in this folder).
```

Figure 21. *A skill: a folder with SKILL.md plus supporting files. Claude loads it automatically when the description matches the task.*

Mechanism	Triggered by	Runs in	Best for
Slash command	The user typing `/name`	Main conversation	Repeatable prompts you invoke on purpose
Skill	Claude, when the `description` matches (or `/name`)	Main conversation (or forked context if configured)	Reusable know-how + files loaded only when relevant
Subagent (`.claude/agents/`)	Claude delegating, or you asking	Its own isolated context window	Verbose exploration, focused reviews, parallel work
Hook	A lifecycle event	Outside the model, as code	Rules that must always run
CLAUDE.md	Always loaded	Every conversation	Persistent project knowledge

ARCHITECT'S NOTE

Skills use **progressive disclosure**: only the name and description sit in context until the skill is needed, then the body and any referenced files load. That is why a skill is better than pasting a long procedure into CLAUDE.md when the procedure is only needed occasionally.

4.5 Plan mode and effective workflows

Plan mode makes Claude research and propose a plan **without modifying anything**. Use it when a change is large, ambiguous, touches many files, or when choosing between architectural approaches. For a one-line typo fix, plan mode is overhead.

Terminal — plan mode TERMINAL

```
> [Shift+Tab twice]  ‖ plan mode on
> migrate session storage from in-memory to Redis
● Research: read src/session/*.ts (4 files), config/, tests/session/ (11 tests)
● Plan:
  1. Add ioredis + RedisSessionStore implementing SessionStore interface
  2. Feature flag SESSION_BACKEND=memory|redis (default memory)
  3. Update 11 tests to run against both backends
  4. Docs: README "Running locally" section
  Risks: TTL semantics differ; serialisation of Date fields.
  Would you like to proceed? 1. Yes, auto-accept edits  2. Yes, manually approve  3. Keep planning
```

Figure 22. *Plan mode: Claude explores and proposes; nothing changes until you approve.*

- **Explore → plan → code → commit** is the recommended loop for non-trivial work.
- **Give Claude a way to verify its work** (tests, type-checks, screenshots) — the single biggest quality lever.
- `/clear` **between unrelated tasks** and `/compact` when a long session fills the window (more in Chapter 6).
- **Be specific:** name files, constraints and acceptance criteria instead of "make it better".

4.6 Claude Code in CI/CD

In a pipeline there is no human to answer prompts, so you run Claude Code **headless** with `-p` (print mode), give it exactly the tools it needs, and ask for machine-readable output.

Terminal — headless review TERMINAL

```
$ claude -p "Review the diff in this PR for bugs and security issues. Output JSON findings." \
    --output-format json \
    --allowedTools "Read,Grep,Glob,Bash(git diff:*)" \
    --max-turns 10
{"type":"result","subtype":"success","is_error":false,"num_turns":6,
 "result":"[{\"file\":\"src/auth.ts\",\"line\":42,\"severity\":\"high\", ...}]",
 "session_id":"5f1c...","total_cost_usd":0.083}
```

Figure 23. *`-p` runs non-interactively; `--output-format json` returns a parseable envelope; `--allowedTools` pre-approves only read-only tools.*

.github/workflows/claude-review.yml EDITOR

```yaml
name: Claude PR review
on:
  pull_request:
    types: [opened, synchronize]
jobs:
  review:
    runs-on: ubuntu-latest
    permissions: { contents: read, pull-requests: write }
    steps:
      - uses: actions/checkout@v4
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
          prompt: "/review  Focus on security and missing tests. Follow CLAUDE.md."
          claude_args: "--max-turns 10 --allowedTools Read,Grep,Glob"
```

Figure 24. *The official GitHub Action. The API key lives in repository secrets; the project CLAUDE.md applies in CI too.*

REAL-WORLD EXAMPLE — CI review that people actually read

A fintech team's CI review bot posted 40 low-value comments per PR and engineers started ignoring it. The fix was not a bigger model. They (1) wrote explicit review criteria in the prompt — *report only bugs, security issues and missing tests; ignore style handled by the linter*; (2) added two examples of a good finding; (3) required JSON output with `severity` and `confidence_reason`; (4) posted only `high`/`medium` findings. Comment volume dropped by 85% and acceptance of suggestions rose sharply.

EXAM TRAP

- Running headless with permission prompts left on hangs the job — pre-approve tools with `--allowedTools` (or settings).
- `--dangerously-skip-permissions` is for isolated sandboxes only, never as the default fix for a CI prompt.
- The Message Batches API is cheaper but asynchronous (up to 24h): fine for nightly bulk reviews, wrong for a blocking pre-merge check.

Chapter 4 check

Q1. A developer put "use pnpm, never npm" in `~/.claude/CLAUDE.md`. New teammates' sessions keep running `npm install`. Fix?

A. Write the rule in capital letters
B. Move the rule to the project's `CLAUDE.md` and commit it
C. Add it to every prompt manually
D. Create a skill called pnpm

Q2. Claude must never read `.env` files, even if asked. Which is the correct control?

A. A `deny` rule such as `Read(./.env)` in project settings
B. A line in CLAUDE.md
C. A skill describing secret handling
D. Plan mode

Q3. Your monorepo's CLAUDE.md is 2,000 lines, mostly testing rules for one package. Responses feel unfocused. Best improvement?

A. Run /compact more often
B. Move package-specific rules to path-scoped rule files or that package's CLAUDE.md, and keep the root file concise
C. Switch to a model with a bigger context window
D. Delete CLAUDE.md

Q4. A pipeline step runs `claude "review this PR"` and the job hangs until timeout. Most likely cause and fix?

A. CLAUDE.md is missing
B. The model is slow; use Haiku
C. The PR is too big; split it
D. It is running interactively; use `-p` with `--output-format json` and pre-approved tools

Q5. A team needs a 15-step data-migration procedure, used about once a month, available to Claude with its templates. Best mechanism?

A. A PreToolUse hook
B. A skill with the procedure and templates in its folder
C. Paste it into the root CLAUDE.md
D. A user-level CLAUDE.md

Chapter 4 answers & explanations

Q1 — Answer B. User-level memory only applies to that one developer. Team conventions belong in the project CLAUDE.md, which is version-controlled and shared.

Q2 — Answer A. Permission deny rules are enforced by the harness. CLAUDE.md and skills are guidance; plan mode only prevents edits.

Q3 — Answer B. Instructions should load only where relevant. Path-scoped rules and subdirectory CLAUDE.md files keep context lean and the important rules prominent.

Q4 — Answer D. Without `-p` Claude Code starts an interactive session waiting for input. Headless mode with explicit tool permissions is required in CI.

Q5 — Answer B. Skills bundle instructions and files and load only when relevant (progressive disclosure), avoiding permanent context cost.

CHAPTER 5 · DOMAIN 4

Exam weight 20%

Prompt Engineering & Structured Output

Explicit criteria, few-shot examples, JSON schemas, validation–retry loops and batch processing.

Learning objectives

- Write prompts with explicit, testable criteria instead of vague adjectives.
- Use few-shot examples to teach format and judgement, including edge cases.
- Obtain reliable structured output with JSON schemas and forced tool use.
- Build validation–retry loops that feed specific errors back to Claude.
- Choose between real-time calls, the Message Batches API and prompt caching for cost and latency.

5.1 Explicit criteria beat adjectives

"Be thorough", "be careful", "only flag important issues" mean different things to different readers — including Claude. Professional prompts define **what counts**, **what does not**, and **what to do in the grey zone**.

Vague (typical wrong answer)	Explicit (what architects write)
"Review this code carefully and report important problems."	"Report only: (1) bugs that produce wrong results, (2) security issues (injection, auth bypass, secrets), (3) missing tests for changed logic. Do NOT report style or naming — the linter handles those. If unsure whether something is a bug, report it with severity `low` and explain the doubt."
"Escalate difficult cases."	"Escalate when: the customer asks for a human; the refund exceeds $500; the policy does not cover the situation; or two attempts to resolve have failed. Do not escalate merely because the customer is upset."

Anatomy of a strong system prompt

prompts/support_system.md EDITOR

```
<role>You are a support resolution agent for Northwind Retail.</role>

<goal>Resolve the customer's issue in this conversation when policy allows (target: 80%
first-contact resolution). Accuracy beats speed.</goal>

<process>
1. Identify the customer with get_customer before discussing any order.
2. Use lookup_order for the specific order. Never guess order details.
3. Apply the refund policy (resource policy://refunds). Quote the relevant clause.
</process>

<escalation_criteria>
Escalate with escalate_to_human if ANY: customer explicitly asks for a human;
amount > $500; policy has no applicable rule; legal threat. Include a case summary.
</escalation_criteria>

<output>Plain language, max 120 words, one clear next step for the customer.</output>
```

Figure 25. *XML tags separate role, goal, process, criteria and output format — easy for Claude to follow and for you to maintain.*

ARCHITECT'S NOTE

Explain the **why** behind rules ("never guess order details — customers act on what we tell them"). Claude generalises better from reasons than from bare commands, and you need fewer ALL-CAPS warnings.

5.2 Few-shot examples

Examples are the most reliable way to communicate format and judgement. Good example sets are **diverse** (cover typical and edge cases), **consistent** (same format) and **wrapped in tags** so they are not confused with instructions.

prompts/review_examples.md EDITOR

```
<examples>
<example>
<code>const total = items.reduce((s, i) => s + i.price * i.qty, 0.0)</code>
<finding>{"severity": "high", "issue": "Float arithmetic on money", "fix": "Use integer cents"}</finding>
</example>
<example>
<code>function calcTot(x) { ... }   // poorly named but correct</code>
<finding>NONE  (naming is out of scope)</finding>
</example>
</examples>
```

Figure 26. *Two examples: one true positive and one deliberate "do not report" case that teaches the boundary.*

EXAM TIP

When a scenario says the model is **inconsistent** in format or keeps flagging the wrong kind of issue, "add a few targeted examples, including a negative example" is frequently the best answer. When it says the model **must** produce valid JSON, the answer is a schema/forced tool, not just examples.

5.3 Structured output: from “please return JSON” to guaranteed shape

Technique	Guarantee	Comment
Ask for JSON in the prompt	None — may add prose, miss fields	OK for prototypes only
Prefill / examples	Better consistency	Still not a guarantee
Forced tool use with `input_schema`	Output arrives as a `tool_use` block shaped by your schema	The classic, widely-tested pattern
Structured outputs (JSON-schema constrained decoding)	Response conforms to the schema	Use where supported for strict schema compliance
Plus your own validation	Business rules (totals add up, dates valid)	Schemas guarantee *shape*, not *truth*

extract/invoice.py EDITOR

```python
INVOICE_TOOL = {
  "name": "record_invoice",
  "description": "Record the fields extracted from one supplier invoice.",
  "input_schema": {
    "type": "object",
    "properties": {
      "invoice_number": {"type": "string"},
      "invoice_date":   {"type": "string", "format": "date"},
      "currency":       {"type": "string", "enum": ["EUR", "USD", "GBP", "OTHER"]},
      "total_cents":    {"type": "integer"},
      "vat_number":     {"type": ["string", "null"],
                         "description": "null if not printed on the invoice. Never invent."},
      "line_items":     {"type": "array", "items": {"$ref": "#/$defs/line"}},
      "extraction_notes": {"type": "string", "description": "Ambiguities, unreadable parts"}
    },
    "required": ["invoice_number", "invoice_date", "currency", "total_cents", "vat_number", "line_items"],
    "$defs": {"line": {"type": "object", "properties": {
        "description": {"type": "string"}, "amount_cents": {"type": "integer"}},
        "required": ["description", "amount_cents"]}}
  }
}

resp = client.messages.create(model=MODEL, max_tokens=2048, tools=[INVOICE_TOOL],
          tool_choice={"type": "tool", "name": "record_invoice"},
          messages=[{"role": "user", "content": [pdf_block, {"type": "text", "text": PROMPT}]}])
data = next(b.input for b in resp.content if b.type == "tool_use")
```

Figure 27. *Schema design choices that prevent hallucination: nullable optional fields, an* `OTHER` *enum value, a notes field, and forced tool use.*

Schema design rules examiners test

- **Make genuinely optional data nullable** (`["string", "null"]`) and say “null if absent — never invent”. A required non-nullable field *forces* the model to fabricate a value when the document lacks one.
- **Give enums an escape hatch** (`OTHER`, `UNKNOWN`) plus a detail field, so unexpected values are not forced into the wrong category.
- **Add a notes / ambiguity field** so the model can report problems instead of hiding them.
- **Use integers for money** and explicit formats for dates.

5.4 Validation–retry loops

Even perfectly shaped JSON can be *wrong*: line items that do not sum to the total, a date in the future. Validate in code, and when validation fails, **send the specific error back** so Claude can correct itself.

extract/validate_retry.py EDITOR

```python
for attempt in range(3):
    data = extract(document, feedback)
    errors = validate(data)                 # schema + business rules
    if not errors:
        return data
    feedback = ("Your previous extraction failed validation:\n- " + "\n- ".join(errors) +
                "\nRe-read the document and correct ONLY these fields. "
                "If the document itself is inconsistent, explain it in extraction_notes.")
route_to_human_review(document, data, errors)   # bounded retries, then a human
```

Figure 28. *Specific feedback, bounded retries, and a human fallback. Blind retries with the same prompt rarely help.*

Terminal — validation in action TERMINAL

```
$ python extract_batch.py invoices/2026-10/
[1/3] INV-88213  ✓ valid
[2/3] INV-88214  × line items sum 1,240.00 ≠ total 1,420.00 -> retry with feedback
[2/3] INV-88214  ✓ valid after retry 1 (total re-read as 1,240.00, OCR '4'/'2' confusion noted)
[3/3] INV-88215  × vat_number missing, document is a receipt -> null accepted, note added
Done: 3 processed, 0 sent to human review
```

Figure 29. *A run showing a retry that fixed a misread total and a correctly-null optional field.*

EXAM TRAP

- Retrying when the information is **not in the document** cannot succeed — make the field nullable or route to a human.
- "Lower the temperature to 0" does not guarantee valid or correct output.
- "Ask the model to double-check its own JSON" is weaker than validating in code.

5.5 Cost and latency: batches and caching

Need	Use	Why
Large volume, results not needed immediately (nightly extraction of 50,000 documents)	**Message Batches API**	About 50% cheaper; processed asynchronously, typically within 24 hours. Match results by `custom_id`.
Interactive or blocking (chat, pre-merge check)	Standard Messages API	Batches have no latency guarantee
Same long prefix reused across calls (big system prompt, policy docs, tool list)	**Prompt caching** (`cache_control`)	Cached reads cost a fraction of normal input and cut latency; default cache lifetime is 5 minutes (1-hour option)
Multi-turn tool loop	Standard API (agent loop)	A batch request is a single request — it cannot run an interactive tool loop

Chapter 5 check

Q1. An extraction schema marks `po_number` as a required string. Many invoices have no PO number, and the model outputs plausible but invented PO numbers. Best fix?

A. Use a larger model
B. Tell the model not to hallucinate
C. Retry until the PO number matches a regex
D. Make `po_number` nullable and instruct: null when absent, never infer

Q2. A nightly job extracts data from 80,000 scanned forms; results are needed by the next morning. Cost is the main concern. Best approach?

A. Claude Code in plan mode
B. Message Batches API with `custom_id` per form
C. One request with all forms concatenated
D. Real-time API with 50 parallel workers

Q3. Validation finds that extracted line items don't sum to the total. What should the retry prompt contain?

A. A higher temperature
B. The specific validation error and an instruction to re-check those fields
C. The same prompt again
D. "Be more careful this time"

Q4. A CI review bot flags too many trivial style issues and developers ignore it. Most effective change?

A. Reduce `max_tokens`
B. Ask for a confidence score and hide anything under 0.7
C. Use a larger model
D. Define explicit in-scope categories, exclude style, and add examples including a 'do not report' case

Q5. Every request to your agent repeats a 30,000-token policy document and tool list. Latency and cost are high. What helps most?

A. Message Batches API
B. Splitting the policy across user messages
C. Removing the system prompt
D. Prompt caching on the stable prefix

Chapter 5 answers & explanations

Q1 — Answer D. A required non-nullable field forces the model to produce something. Allowing null gives it a correct way to say "absent".

Q2 — Answer B. Non-urgent, high-volume work is the textbook case for the Batches API (about half the cost). `custom_id` maps each result back to its form.

Q3 — Answer B. Targeted feedback tells Claude exactly what to fix. Identical or generic retries usually reproduce the same mistake.

Q4 — Answer D. Precision problems come from vague criteria. Explicit categories plus a negative example address the root cause; self-reported confidence is poorly calibrated.

Q5 — Answer D. Prompt caching reuses an identical prefix across calls, cutting both cost and latency; batches would add latency to an interactive agent.

CHAPTER 6 · DOMAIN 5

Exam weight 15%

Context Management & Reliability

Keeping critical facts alive in long sessions, escalation and human-in-the-loop design, provenance, and error handling at scale.

Learning objectives

- Explain why long contexts degrade and how to preserve critical facts.
- Use compaction, scratchpad/state files, subagents and `/clear` to manage the context window.
- Design escalation rules and human-in-the-loop checkpoints that are explicit and auditable.
- Preserve provenance (citations, sources) through multi-step and multi-agent pipelines.
- Handle failures gracefully: partial results, retries, and honest reporting.

6.1 Why context is a budget, not a bucket

A bigger window does not mean everything inside it is used equally well. As history grows, details from early turns can be overlooked, verbose tool results crowd out instructions, and summaries can silently drop numbers, dates and IDs. Architects treat context as a **scarce budget** to spend on the information that matters for the next decision.

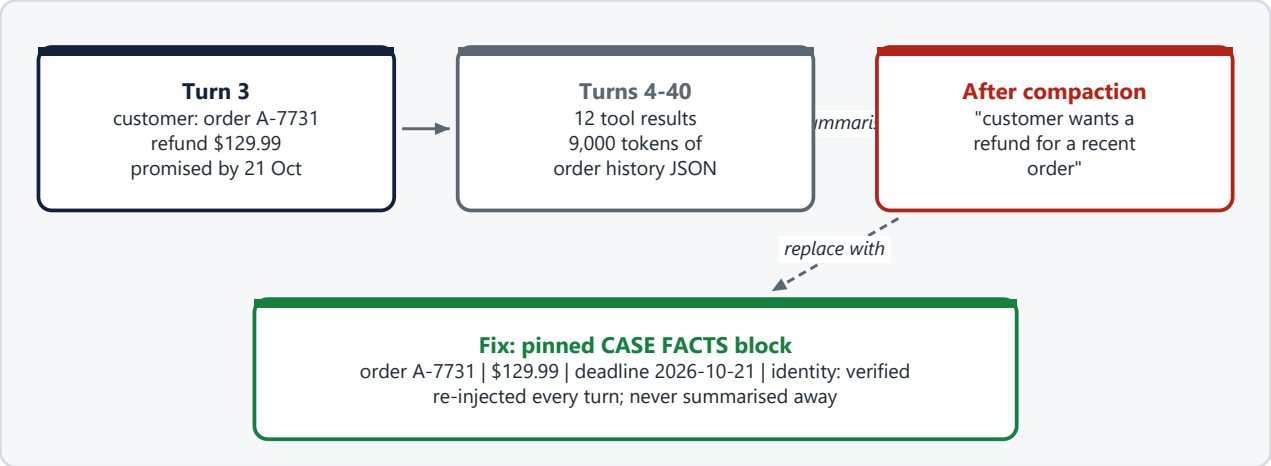


Figure 30. *How a precise fact decays into a vague summary — and the pattern that prevents it.*

6.2 Techniques for keeping what matters

Technique	How	Use when
Case-facts / state block	Keep critical facts (IDs, amounts, dates, decisions) in a structured block maintained by code and included every turn	Long customer conversations, any multi-step task with key numbers
Trim tool results	Return only fields the agent needs (PostToolUse hook or tool design); paginate	Tools that return large JSON payloads
Scratchpad / progress files	Agent writes plans, findings and progress to files (e.g. `progress.md`, a todo list) and re-reads them	Long-running tasks, work spanning sessions or compactions
Subagents	Delegate verbose exploration; receive a condensed result	Reading many files or sources
Compaction	Summarise old history (`/compact` in Claude Code, automatic near the limit) with instructions on what to keep	Long sessions — but tell it what must survive
Fresh start + summary	`/clear` or a new session seeded with a structured summary	Switching tasks, or when the context is stale
Position matters	Put long documents first and the question/instructions after them	Long-document Q&A

Terminal — Claude Code context hygiene TERMINAL

```
> /context
  Context usage: 148k / 200k tokens (74%)
  System prompt 3.1k | Tools 11.4k | MCP tools 18.9k | Memory files 2.2k | Messages 112.4k
> /compact keep the list of failing test names, the root cause hypothesis and the files changed
● Compacted conversation. Preserved: 7 failing tests, hypothesis (race in SessionStore.flush), 4 files.
> /clear
  (new task: start fresh - the migration work is unrelated)
```

Figure 31. *Inspect what is using the window, compact with explicit instructions about what to preserve, and clear between unrelated tasks.*

> **ARCHITECT'S NOTE**
> Notice that MCP tool definitions alone took 18.9k tokens above. Connecting every available MCP server "just in case" has a real context cost — another reason to scope tools per project and per agent.

6.3 Escalation and human-in-the-loop

Reliable systems know their limits. Escalation must be triggered by **explicit, observable criteria**, and the hand-off must carry enough context that the human does not have to start over.

Good escalation triggers	Poor escalation triggers
Customer explicitly requests a human	Model's self-reported confidence < X
Action exceeds an authority limit (amount, account change)	Negative sentiment alone

Good escalation triggers	Poor escalation triggers
Policy does not cover the situation	"The case seems complex"
Repeated failed attempts (e.g. 2 tool failures on the same step)	Every refund, "to be safe" (destroys resolution rate)
Legal, safety or fraud signals	Random sampling as the only control

escalate_to_human — tool input EDITOR

```
{
  "case_id": "C-20261022-0193",
  "customer_id": "CU-55102",  "identity_verified": true,
  "issue": "Damaged item, order A-7731, refund requested $629.00",
  "reason_for_escalation": "amount_over_limit",
  "actions_taken": ["verified identity", "confirmed delivery 2026-10-18", "photo received"],
  "policy_reference": "refunds.md §3.2 (damaged on arrival)",
  "recommended_action": "approve full refund",
  "customer_expectation": "told a specialist will reply within 4 business hours"
}
```

Figure 32. *A structured hand-off: the human sees what happened, why it was escalated and what was promised.*

EXAM TRAP

"Escalate when the model's confidence score is below 0.8" looks scientific but is a frequent wrong answer: LLM self-assessed confidence is not reliably calibrated. Prefer explicit rules and deterministic checks.

6.4 Provenance: keeping sources attached to claims

In research and extraction pipelines, every claim should remain traceable to its source. Provenance is lost when intermediate steps return free-text summaries. Preserve it by **making citations part of the data structure** at every hop.

subagent finding (structured) EDITOR

```
{"claim": "EU heat-pump sales fell 21% in 2025",
 "evidence_quote": "...sales declined by 21% year-on-year across 14 markets...",
 "source_url": "https://example-association.eu/report-2026.pdf",
 "published": "2026-02-11",
 "confidence_basis": "single industry source; no independent confirmation",
 "conflicts_with": null}
```

Figure 33. *Claim + quote + URL + date travels together from search subagent to synthesiser to final report.*

- When sources **conflict**, surface both with their dates rather than silently picking one.
- Prefer recent, primary sources; record the publication date.
- Ask the synthesiser to mark claims supported by a single source.

6.5 Reliability under failure

- **Retry transient errors with backoff** (ideally inside the tool), but cap retries.
- **Partial success is reported as partial:** "7 of 9 sources retrieved; 2 timed out" — never presented as complete.

- **Idempotency** for write tools (refunds, emails): use idempotency keys so a retry does not refund twice.
- **Observability:** log tool calls, inputs, outputs, hook decisions and `stop_reason` per turn so failures can be audited.
- **Evaluate continuously:** a test set of real cases (including edge cases) run on every prompt or tool change.

> **REAL-WORLD EXAMPLE — Summaries that changed the numbers**
> An insurance claims assistant summarised 60-turn conversations to save tokens. Auditors found that in 4% of cases the summary changed the claimed amount or dropped the incident date. The fix: a code-maintained `claim_facts` block (policy number, amounts, dates, decisions) injected on every turn and excluded from summarisation, plus a PreCompact hook that writes it to the database. Discrepancies dropped to zero in the next audit.

Chapter 6 check

Q1. In long support chats, the agent sometimes refers to the wrong order amount after the conversation is summarised. Best fix?

A. Maintain a structured case-facts block (IDs, amounts, dates) included on every turn
B. Use a model with a larger context window
C. Stop summarising and let the context overflow
D. Ask the customer to repeat the amount

Q2. Which escalation rule is most appropriate for a support agent?

A. Escalate when the customer explicitly asks for a human or the refund exceeds the agent's authority
B. Escalate whenever confidence is below 0.7
C. Escalate every conversation with negative sentiment
D. Never escalate; the target is 100% resolution

Q3. A research system's final reports contain correct facts but missing or wrong citations. Where is provenance most likely lost?

A. In the final PDF rendering
B. When subagents return free-text summaries without source fields
C. In the web search tool
D. In the user's question

Q4. A Claude Code session has been debugging for hours; you now want to start an unrelated feature. Best practice?

A. Continue in the same session
B. Switch to plan mode
C. Increase the max context
D. Use `/clear` (optionally after saving notes) and start fresh

Q5. A document-processing agent retrieves 9 sources, 2 of which time out after retries. What should the final output do?

A. Invent likely content for the missing sources

B. State that 7 of 9 sources were used and list the 2 unavailable ones

C. Present the result as complete

D. Fail the entire job

Chapter 6 answers & explanations

Q1 — Answer A. Critical facts must not depend on a lossy summary. A code-maintained facts block keeps them exact and always visible.

Q2 — Answer A. Explicit, observable triggers are reliable and auditable. Self-confidence is poorly calibrated; sentiment alone is a weak signal.

Q3 — Answer B. If intermediate outputs do not carry claim–source pairs as structured data, later stages cannot reconstruct them.

Q4 — Answer D. Unrelated old context wastes tokens and can mislead. Clear it; persist anything important to files first.

Q5 — Answer B. Reliable systems report partial results honestly so downstream users can judge coverage.

CHAPTER 7

The Six Exam Scenarios, Solved

Each scenario as an architect would design it, with the decisions and traps you will meet on exam day.

You will see four of these six scenarios. Each brings together several domains, so use this chapter as integrated revision: if you can explain every decision table below aloud, you are ready.

Scenario 1: Customer Support Resolution Agent

Domains: D1 Agentic Architecture · D2 Tools & MCP · D5 Reliability

The situation

An online retailer wants an Agent SDK agent handling returns, billing disputes and account issues through MCP tools `get_customer`, `lookup_order`, `process_refund` and `escalate_to_human`. Target: 80%+ first-contact resolution, without ever violating refund policy.

Reference architecture

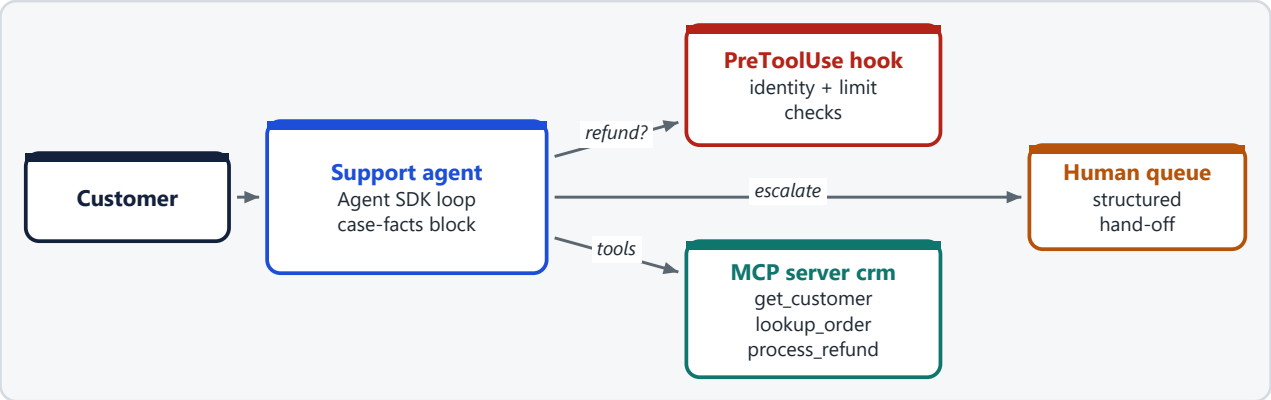


Figure 34. *Scenario 1 reference architecture.*

Decisions the exam will ask you to justify

Question	Architect's answer
How do we guarantee identity is verified before any refund?	A programmatic prerequisite / PreToolUse hook that denies `process_refund` until `get_customer` has verified identity in this session.
The agent picks `lookup_order` when it should call `get_customer`.	Rewrite both tool descriptions with clear purpose, order of use and boundaries.
When should it escalate?	Explicit triggers: customer asks for a human, amount over authority, no applicable policy, repeated failures. Not sentiment or self-confidence.
Long chats lose the order amount.	Pinned case-facts block maintained by code.

Question	Architect's answer
The CRM sometimes times out.	Structured transient error with `retryable: true`; bounded retry; honest message or escalation.

EXAM TRAP

- Escalating everything "to be safe" — kills the 80% resolution goal.
- Relying on prompt wording for the $ limit.
- Using a confidence score as the escalation gate.

Scenario 2: Code Generation with Claude Code

Domains: D3 Claude Code · D5 Context

The situation

A 30-developer team adopts Claude Code on a TypeScript monorepo. They want consistent conventions, shared workflows and safe defaults, while each developer keeps personal preferences.

Reference architecture

Project `CLAUDE.md` (commands, architecture, conventions) + `.claude/rules/` path-scoped rules per package + `.claude/commands/` and `.claude/skills/` for team workflows + `.claude/settings.json` permissions committed to git. Personal preferences stay in `~/.claude/CLAUDE.md` and `settings.local.json`.

Decisions the exam will ask you to justify

Question	Architect's answer
Where do team conventions go?	Project CLAUDE.md (committed). Never user-level.
Rules only for the `payments` package?	`packages/payments/CLAUDE.md` or a rule file with `paths: packages/payments/**`.
A repeatable "fix issue" workflow?	A project slash command or skill in `.claude/` shared via git.
A large refactor across 40 files?	Plan mode first; review the plan; then execute with tests as verification.
Claude must never touch `infra/prod/`.	Deny permission rules (or a hook), not a CLAUDE.md sentence.

EXAM TRAP

- Putting secrets or personal paths in the committed CLAUDE.md.
- One giant CLAUDE.md for everything.
- Using plan mode for trivial one-line edits.

Scenario 3: Multi-Agent Research System

Domains: D1 Agentic Architecture · D2 Tools · D5 Provenance

The situation

A coordinator delegates to web-search, document-analysis, synthesis and report-writing subagents to produce comprehensive, cited reports on market questions.

Reference architecture

Hub-and-spoke (see Figure in Chapter 2). The coordinator decomposes the question into independent sub-questions, runs search subagents **in parallel** with explicit briefs, receives **structured findings** (claim, quote, URL, date), and passes them to a synthesiser that has no search tools. Optionally an evaluator subagent checks every claim has a source.

Decisions the exam will ask you to justify

Question	Architect's answer
Subagents duplicate each other's searches.	Coordinator assigns distinct, non-overlapping sub-questions in explicit briefs.
Citations disappear in the final report.	Make claim–source pairs part of the output schema at each hop.
Coordinator's context overflows.	Subagents return condensed, structured findings — not raw pages.
Sources disagree.	Report both with dates; do not silently choose.
One search subagent fails.	Continue with partial results and disclose the gap, or retry that sub-question.

EXAM TRAP

- Sharing the full coordinator history with every subagent.
- Letting subagents call each other directly.
- Giving the synthesiser web search "to fill gaps" (it will invent new uncited claims).

Scenario 4: Developer Productivity with Claude

Domains: D2 Tools & MCP · D1 Architecture · D3 Claude Code

The situation

Engineers use Claude (Claude Code / Agent SDK) with built-in tools and MCP servers to understand an unfamiliar 2-million-line codebase, answer questions about it and make changes.

Reference architecture

Built-in `Grep`/`Glob`/`Read` for code search; an **Explore-style subagent** for broad investigation that returns summaries; MCP servers for issue tracker and docs; CLAUDE.md files documenting architecture so exploration starts informed.

Decisions the exam will ask you to justify

Question	Architect's answer
How should Claude find where a function is used?	`Grep` for the symbol, then `Read` the relevant files — targeted, not reading whole directories.
Investigation floods the main context.	Delegate exploration to a subagent that returns a concise summary.

Question	Architect's answer
Connecting 12 MCP servers slowed everything down.	Tool definitions cost context: enable only the servers needed for this project.
Choosing between a built-in tool and an MCP tool that overlap?	Clear descriptions; remove or scope the redundant one.

EXAM TRAP

- "Load the whole repository into context."
- Writing a custom MCP server for something a built-in tool already does.

Scenario 5: Claude Code for Continuous Integration

Domains: D3 Claude Code · D4 Prompt Engineering

The situation

The platform team wants automated PR review, test generation for uncovered code, and actionable feedback in the pipeline — without noisy comments or unsafe permissions.

Reference architecture

`anthropics/claude-code-action` or `claude -p` in the pipeline; `--output-format json`; read-only `--allowedTools` for review; explicit review criteria with examples; JSON findings filtered by severity before posting; API key in CI secrets; project CLAUDE.md applies automatically.

Decisions the exam will ask you to justify

Question	Architect's answer
The job hangs.	It is interactive; use `-p` and pre-approve tools.
Too many trivial comments.	Explicit criteria, exclude style, add negative examples, post only high/medium.
Need machine-readable output.	`--output-format json` and a defined findings schema.
Nightly review of all repos, cost-sensitive?	Batch processing is fine here — nothing is blocked waiting.
The same review must run before merge.	Synchronous call; batches are too slow for a blocking check.

EXAM TRAP

- `--dangerously-skip-permissions` on a shared runner.
- A reviewer agent with write/push permissions it does not need.

Scenario 6: Structured Data Extraction

Domains: D4 Structured Output · D5 Reliability

The situation

A logistics firm extracts fields from invoices, bills of lading and customs forms (PDF scans, many layouts) into a database, with JSON-schema validation and correct handling of missing or ambiguous data.

Reference architecture

Forced tool use (or structured outputs) with a carefully designed schema → code validation (schema + business rules) → retry with specific error feedback (max 2–3) → human review queue. Batches for bulk back-office loads; few-shot examples for tricky layouts.

Decisions the exam will ask you to justify

Question	Architect's answer
Model invents missing values.	Nullable fields + "null if absent" + notes field.
Unexpected document type.	Enum with `OTHER` + detail field, or route to a human.
Totals don't add up.	Business-rule validation in code; retry with the specific discrepancy.
50,000 documents nightly.	Message Batches API with `custom_id`.
Inconsistent date formats.	Schema `format: date` + an example; normalise in code.

EXAM TRAP

- "Ask for JSON in the prompt" as the production solution.
- Infinite retries on a field that is not in the document.
- Trusting schema-valid output as correct output.

CHAPTER 8

Full Mock Exam — 30 Questions

Timed practice in the exam's style. Allow 60 minutes. Score yourself with the answer key that follows.

EXAM TIP

Simulate the real thing: no notes, a timer at 60 minutes, answer every question. Target **24/30 (80%)** or better before booking — it gives a safety margin over the 720 pass mark.

Q1. [Scenario 1] After each tool call, your loop appends only the tool results, not the assistant message. The next API call fails. Why?

A. The system prompt must be resent as a user message

B. `max_tokens` is too low

C. The tool results are too large

D. Each `tool_result` must follow the assistant message containing the matching `tool_use` block

Q2. [Scenario 1] Claude returns two `tool_use` blocks in one response. How should you send the results?

A. One user message containing both `tool_result` blocks

B. Only the first result; Claude will ask again

C. Merge both into a single text block

D. Two separate user messages, one per result

Q3. [Scenario 1] The business requires that `process_refund` is never called for unverified customers. The prompt already says so. Remaining failure rate: 2%. What closes the gap?

A. A post-hoc audit each week

B. A programmatic prerequisite/hook that blocks the call until verification is recorded

C. More few-shot examples

D. Lower temperature

Q4. [Scenario 1] Which escalation design best balances the 80% resolution target with safety?

A. Escalate when self-rated confidence < 0.9

B. Escalate on explicit request, over-limit amounts, policy gaps, or repeated failures

C. Escalate when sentiment is negative

D. Escalate all refunds

Q5. [Scenario 1] `lookup_order` fails with a database timeout. What should the tool return?

A. A generic "Error" string

B. `is_error: true` with a structured transient error and `retryable: true`

C. An empty object

D. Nothing — let the loop time out

Q6. [Scenario 2] A developer wants Claude to always answer them in French, but the team works in English. Where should this preference go?

A. Project CLAUDE.md

B. `.claude/settings.json`

C. A team slash command

D. `~/.claude/CLAUDE.md`

Q7. [Scenario 2] Testing conventions should apply only when Claude edits `*.test.ts` files. Best approach?

A. Ask developers to paste them when needed

B. A PreToolUse hook

C. Add them to the root CLAUDE.md

D. A `.claude/rules/` file with a `paths` pattern for test files

Q8. [Scenario 2] A teammate asks Claude to restructure the authentication module across 25 files. What should come first?

A. A new MCP server

B. Accept-edits mode to move fast

C. `/clear`

D. Plan mode to research and propose a plan for review

Q9. [Scenario 2] Which file should hold permission rules shared by the whole team?

A. `CLAUDE.local.md`

B. `.claude/settings.json`

C. `~/.claude/settings.json`

D. `.claude/settings.local.json`

Q10. [Scenario 3] Two search subagents return overlapping results while an important angle is never researched. Root cause?

A. Web search is unreliable

B. Too few subagents

C. The coordinator's task decomposition and briefs are not distinct and complete

D. Subagents use the wrong model

Q11. [Scenario 3] How should a search subagent return its work to the coordinator?

A. Its entire conversation transcript

B. The raw HTML of every page

C. A single paragraph of prose

D. A condensed, structured list of findings with claim, quote, URL and date

Q12. [Scenario 3] The synthesis subagent occasionally adds facts that no source supports. Best fix?

A. Raise temperature for creativity

B. Restrict it to provided findings, require a citation for every claim, and add an evaluator check

C. Use a larger model

D. Give it web search to verify

Q13. [Scenario 3] Two reputable sources give different market sizes. What should the report do?

A. Present both with sources and dates and note the discrepancy
B. Omit the figure
C. Average them
D. Choose the larger figure

Q14. [Scenario 4] Claude needs to find every caller of `calculateTax` in a large repo. Most efficient approach?

A. Use WebSearch
B. Ask the user to list the files
C. Read every file in `src/`
D. `Grep` for the symbol, then `Read` only matching files

Q15. [Scenario 4] After enabling 15 MCP servers, the team notices slower responses and worse tool choices. Why, and what fix?

A. Every tool definition consumes context and adds selection ambiguity; enable only needed servers per project
B. The model needs fine-tuning
C. Network latency; use a VPN
D. MCP is incompatible with Claude Code

Q16. [Scenario 4] A team needs read-only access to internal design docs for Claude, loaded as context on demand. Which MCP primitive fits best?

A. Tool
B. Sampling
C. Prompt
D. Resource

Q17. [Scenario 5] Your CI step must post PR review comments from Claude's output. Which flags matter most?

A. `--verbose`
B. `-p` with `--output-format json`
C. `--model opus`
D. `--continue`

Q18. [Scenario 5] The review job must not modify code. How do you guarantee it?

A. Tell Claude not to edit in the prompt
B. Use plan mode in CI
C. Allow only read-only tools (e.g. Read, Grep, Glob) via `--allowedTools` / permissions
D. Run it on a weekend

Q19. [Scenario 5] The team wants a full re-review of 400 repositories every Sunday at minimum cost. Best choice?

A. Real-time API
B. Interactive Claude Code sessions
C. Message Batches API
D. Prompt caching only

Q20. [Scenario 5] The bot produces many false positives. Which change addresses the root cause?

A. Post only comments with model confidence ≥ 0.8
B. Use a larger model
C. Define explicit review criteria with in/out-of-scope examples
D. Limit to 3 comments per PR

Q21. [Scenario 6] The schema requires `delivery_date` but some documents have none. The model invents dates. Fix?

A. Drop the field
B. Make the field nullable and instruct "null when not present"
C. Use today's date as default
D. Retry until a date appears

Q22. [Scenario 6] Extracted JSON is schema-valid but totals are sometimes wrong. What adds the most reliability?

A. Business-rule validation in code with targeted retry feedback
B. A longer prompt
C. Nothing — schema-valid means correct
D. Temperature 0

Q23. [Scenario 6] How do you force Claude to return extraction results through `record_invoice`?

A. `stop_sequences: ["}"]`
B. A system prompt instruction
C. `tool_choice: {"type": "any"}`
D. `tool_choice: {"type": "tool", "name": "record_invoice"}`

Q24. [Scenario 6] A new document type appears that fits no `doc_type` enum value. What should the schema allow?

A. An `OTHER` value plus a free-text detail field
B. Nothing; reject it silently
C. The closest existing value
D. Free text for every field

Q25. [General] Which statement about subagents is correct?

A. They inherit the parent's full conversation
B. They can only use MCP tools
C. They run in their own context and return a result to the parent
D. They share a single context window with the coordinator

Q26. [General] Which hook is best for redacting PII from tool results before Claude sees them?

A. PostToolUse
B. PreToolUse
C. UserPromptSubmit
D. SessionEnd

Q27. [General] A response has `stop_reason: "max_tokens"`. What does it mean?

A. The context window is full of input
B. Claude finished normally
C. The output was cut off at your token limit
D. Claude wants a tool

Q28. [General] When is a deterministic workflow better than an autonomous agent?

A. Whenever the task uses tools
B. Never
C. When steps are known and predictability, cost and auditability matter
D. When steps are unknown in advance

Q29. [General] Your agent re-sends a 20k-token system prompt on every turn. What reduces cost and latency without changing behaviour?

A. Removing tools
B. Shortening the user messages
C. Prompt caching
D. Using the Batches API

Q30. [General] Which is the strongest reason to split one agent with 25 tools into role-specific agents?

A. Smaller, focused tool sets improve selection accuracy and enforce least privilege
B. Agents are cheaper per token
C. The API forbids more than 20 tools
D. Subagents always run faster

Mock exam answer key

Q1 — Answer D. Tool results reference `tool_use_id`s that must appear in the preceding assistant turn. Always append the full assistant content first.

Q2 — Answer A. Parallel tool calls are answered together in a single user message with one `tool_result` per `tool_use_id`.

Q3 — Answer B. Only deterministic enforcement turns 98% into 100% for a hard rule.

Q4 — Answer B. Explicit, observable criteria protect the business without over-escalating.

Q5 — Answer B. Structured, categorised errors let Claude choose: retry, tell the customer, or escalate.

Q6 — Answer D. Personal preferences belong at user level so they do not affect teammates.

Q7 — Answer D. Path-scoped rules load only for matching files, keeping context focused.

Q8 — Answer D. Large, risky, multi-file changes benefit from a reviewed plan before any edit.

Q9 — Answer B. Project settings are committed and shared; `.local` and user files are personal.

Q10 — Answer C. Coverage problems in multi-agent systems usually trace back to decomposition: clear, non-overlapping briefs fix them.

Q11 — Answer D. Condensed structured output protects the coordinator's context and preserves provenance.

Q12 — Answer B. Scope tools and inputs tightly and verify citations; giving it search encourages new uncited material.

Q13 — Answer A. Transparent reporting of conflicts preserves trust and lets readers judge.

Q14 — Answer D. Targeted search tools minimise context use and time.

Q15 — Answer A. Tool sprawl costs tokens and accuracy. Scope servers to the task.

Q16 — Answer D. Resources expose read-only, URI-addressable data controlled by the application.

Q17 — Answer B. Headless mode plus JSON output gives a parseable result in a pipeline.

Q18 — Answer C. Tool permissions are enforced by the harness; the prompt is not.

Q19 — Answer C. Non-urgent bulk work suits batch processing at roughly half the cost.

Q20 — Answer C. Precision comes from explicit criteria and examples, not self-reported confidence or arbitrary caps.

Q21 — Answer B. Allowing null removes the pressure to fabricate.

Q22 — Answer A. Schemas guarantee shape, not truth. Validate semantics in code.

Q23 — Answer D. Forcing the named tool guarantees a `tool_use` block for that schema.

Q24 — Answer A. An escape hatch prevents forced misclassification and surfaces new types.

Q25 — Answer C. Context isolation is the defining feature of subagents.

Q26 — Answer A. PostToolUse runs after the tool returns and can transform or filter its output.

Q27 — Answer C. The answer is truncated. Increase `max_tokens`, continue the generation, or split the task.

Q28 — Answer C. Use the simplest architecture that meets the requirement; agents are for open-ended paths.

Q29 — Answer C. Caching a stable prefix is designed exactly for this.

Q30 — Answer A. Focused tool sets reduce confusion and limit the blast radius of mistakes.

EXAM TIP — Score yourself

Notice the pattern? In most questions the correct answer is the one that (1) uses a deterministic control for a hard rule, (2) fixes the root cause at the right layer, or (3) keeps context and tools narrow. That pattern is the exam.

On the real exam, correct answers are distributed across A–D — do not rely on letter patterns, rely on reasoning.

Your score	What it means	What to do
27–30	Exam-ready	Book the exam; do a light review of traps the week before
24–26	Likely pass	Re-read the chapters behind every miss; repeat the labs
18–23	Borderline	Spend another week on hands-on labs for your two weakest domains
< 18	Not yet	Follow the 4-week plan in Chapter 9 from the start

CHAPTER 9

Your 4-Week Pass Plan & Exam Day

A realistic schedule for a beginner studying 1–2 hours a day, plus the strategy for the day itself.

Week	Focus	Hands-on (non-negotiable)	Done when…
1	Ch. 0–2: basics, agent loop, multi-agent, hooks (D1)	Build the loop from Fig. 4; add a second tool and an error; port to Agent SDK with a PreToolUse hook	You can explain every `stop_reason` and why hooks beat prompts for hard rules
2	Ch. 3–4: tools, MCP, Claude Code (D2, D3)	Write a FastMCP server; add it at project scope; create CLAUDE.md, one rule file, one command, one skill; run `claude -p` with JSON output	You can place any instruction at the right CLAUDE.md/settings level
3	Ch. 5–6: prompting, structured output, context (D4, D5)	Build the invoice extractor with a schema, validation and retry; try `/context` and `/compact` in a long session	You can design a schema that cannot force hallucinations
4	Ch. 7–8: scenarios and mock exam	Explain each scenario's decision table aloud; take the mock exam timed; review every miss	Mock score ≥ 24/30, then book

Official resources to add to your plan

- **The official exam guide** on Anthropic's certification page — domain task statements and sample questions.
- **Anthropic Academy** courses (Claude API, Agent SDK / building agents, Claude Code in Action, MCP introduction and advanced topics).
- **docs.claude.com** — Messages API, tool use, structured outputs, prompt engineering guide, Agent SDK, Claude Code docs (memory, settings, hooks, skills, subagents, headless mode).
- **Anthropic engineering blog** — *Building effective agents*, *Writing tools for agents*, *Effective context engineering for AI agents*, the multi-agent research system write-up, and *Claude Code best practices*.
- **modelcontextprotocol.io** — MCP specification and SDKs.

Exam-day strategy

1. **Before:** test your online-proctoring setup the day before; clear your desk; have ID ready.
2. **First pass (≈ 70 min):** answer every question you are confident about; flag the rest. Do not spend more than 3 minutes on any one question.
3. **Read the scenario framing once, carefully.** Many questions in the same scenario reuse its constraints (limits, targets, compliance needs).
4. **For every question:** find the constraint → guarantee or guidance? → smallest root-cause fix → eliminate the usual suspects.

5. **Second pass (≈ 40 min):** revisit flagged questions. Change an answer only for a concrete reason.
6. **Never leave a blank** — there is no guessing penalty.

KEY TAKEAWAYS — The one-page cheat sheet

- Hard rule → hook / permission / prerequisite / schema. Preference → prompt.
- Wrong tool chosen → fix the description. Wrong format → examples/schema. Wrong facts → nullable fields + validation.
- Loop control → `stop_reason`. Turn limits are safety nets.
- Subagents get explicit briefs and minimal tools; they return condensed, structured results.
- Team config → project files in git; personal → user/local files; secrets → env vars.
- Bulk + not urgent → Batches. Repeated prefix → caching. Blocking/interactive → standard API.
- Escalate on explicit criteria, never on self-reported confidence.
- Critical facts live in a maintained block, not in summaries. Provenance travels as structured data.

APPENDIX

Glossary

Terms you must recognise instantly on exam day.

Term	Definition
Agent	A model using tools in a loop, deciding its own next steps until a goal is met.
Agent SDK (Claude Agent SDK)	Anthropic's Python/TypeScript library providing the agent loop, built-in tools, hooks, subagents, sessions and MCP support.
Batches API	Asynchronous bulk processing of many independent Messages requests at reduced cost, results within 24 h.
CLAUDE.md	Markdown memory file automatically loaded by Claude Code at enterprise, user, project or subdirectory level.
Compaction	Summarising older conversation history to free context space (`/compact`).
Context window	The maximum tokens a model can consider at once: system prompt, tools, history, results and output.
Coordinator (orchestrator)	The lead agent that decomposes a task, delegates to subagents and merges their results.
Few-shot examples	Example inputs/outputs included in the prompt to teach format and judgement.
Headless mode	Running Claude Code non-interactively with `claude -p`, typically in scripts and CI.
Hook	User code executed at a lifecycle event (PreToolUse, PostToolUse, Stop…) to enforce or observe behaviour deterministically.
`is_error`	Flag on a `tool_result` signalling that the tool call failed.
MCP (Model Context Protocol)	Open protocol connecting AI hosts to servers that expose tools, resources and prompts.
Permission rules	Allow / ask / deny rules in Claude Code settings controlling which tools and commands may run.
Plan mode	Claude Code mode in which Claude researches and proposes a plan without making changes.
Prompt caching	Reusing a processed prompt prefix across requests to reduce cost and latency.
Provenance	The traceable link between a claim and its source.
Skill	A folder with SKILL.md (and optional files) that Claude loads when its description matches the task.
Slash command	A user-invoked saved prompt (`/name`), e.g. stored in `.claude/commands/`.
`stop_reason`	Response field explaining why generation stopped: `end_turn`, `tool_use`, `max_tokens`, `stop_sequence`, `pause_turn`, `refusal`.

Term	Definition
Subagent	A delegated agent with its own context window, prompt and tool set that returns a result to its parent.
`tool_choice`	Request parameter controlling tool use: `auto`, `any`, `tool` (forced), `none`.
`tool_use` / `tool_result`	The content blocks for Claude's tool request and your response, linked by `tool_use_id`.
Workflow	A predefined, code-orchestrated sequence of LLM calls (chaining, routing, parallelisation…).

Good luck — and remember: the exam rewards the engineer who chooses the right layer for every decision. You now know what those layers are. Go build something with them.